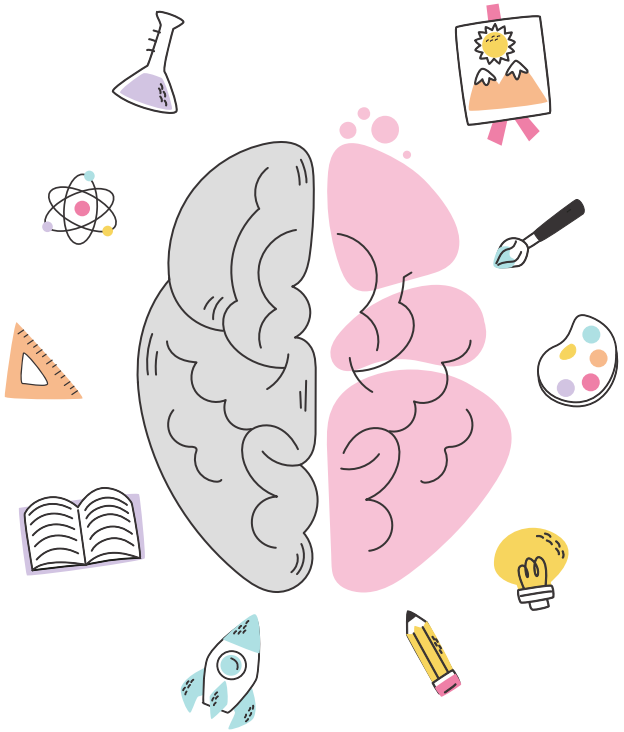
NeuroVibe



Anusha[RA2311003010584],
Koushani[RA2311003010585],
Rishit [RA2311003010587]

Recap of Review-1

Problem Statement:

Physically challenged individuals or users with limited mobility need intuitive ways to control electrical appliances.

Objective:

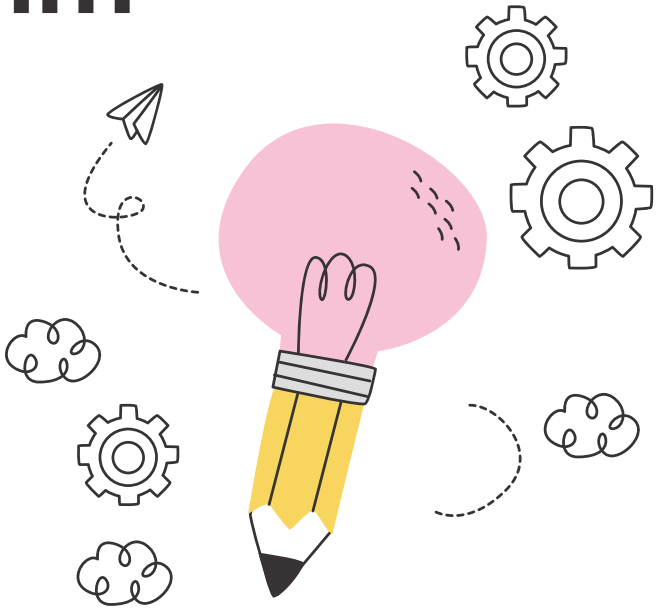
To build a low-cost brain–computer interface that allows users to control room devices (light, fan) by detecting EEG patterns corresponding to mental “ON/OFF” commands.

Proposed System Idea:

Capture EEG signals using Brain BioAmp Band + BioAmp EXG PillProcess & classify signals with a machine-learning modelSend results to ESP32 for IoT-based device control (LED / DC motor)

Mathematical Modelling of the Algorithm

Input Variables: EEG signal windows $X \in \mathbb{R}^n$ (n samples per window)
Output Variable: $Y \in \{0 = LED\_OFF, 1 = LED\_ON\}$
Feature Extraction Equations:
•Mean $\mu = (1/n) \Sigma x_i$
•Std $\sigma = \sqrt{(1/n \Sigma (x_i - \mu)^2)}$
•Band-power features via FFT: $P_k = |FFT_k(X)|^2$
Model Equation (Random Forest):
$f(X) = \text{argmax} \Sigma w_j \cdot I_j(X)$ (over decision trees j)
r
Optimization: Information Gain / Gini Impurity minimization in trees



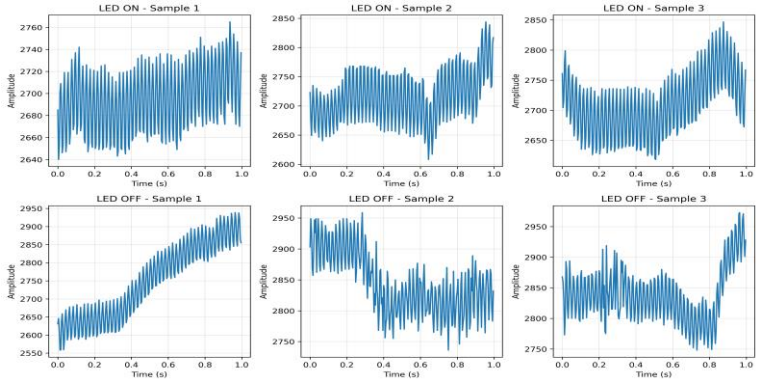
Initial / Partial Results

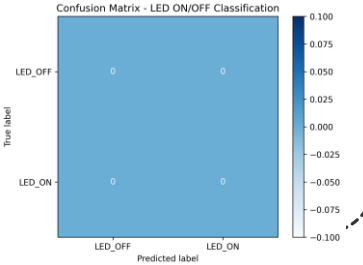
Metric	Value
Accuracy	72 %
Precision	0.70
Recall	0.73
F1-Score	0.71

Observations:

Clear separation between “LED ON” and “LED OFF” brain states in filtered signals. Accuracy improves with longer, cleaner recordings. Real-time test successfully toggled LED on ESP32 when classified > 0.7 confidence.

Graphs / Tables:





Python Implementation of the Model

- **Key Scripts:** record_eeg.py, train_model.py, run_realtime.py
- **Implementation Flow:**
  1. Collect raw EEG → NumPy arrays
  2. Pre-process (filter 1–50 Hz, normalize)
  3. Extract statistical + FFT features
  4. Train Random Forest classifier
  5. Send real-time predictions to ESP32 via serial
- Sample Snippet:

```
from sklearn.ensemble import RandomForestClassifier
import joblib, numpy as np


data = np.load('led_dataset.npz')
X, y = data['X'], data['y']
model = RandomForestClassifier(n_estimators=100)
model.fit(X.reshape(len(X), -1), y)
joblib.dump(model, 'eeg_model.joblib')
```

Sample Visualization: